

Core JS + Node + MongoDB Test

Practical Assessment: 30 Questions

Date: 7-Oct

Marks: 100

Time: 1hr:30min

Part 1: Core JS + Node Test

1. Variables and Console

Write a Node.js program to:

- Store a product name, price, and quantity in stock.
- Display them using `console.log()`.

2. Loop Program

Print the even numbers from 1 to 100 using any one loop.

3. Array Operations

Create an array of movie names. Perform:

- Add element
- Remove element
- Display all elements

4. OS Module

Display:

- Home directory
- OS type
- Host Name
- Free memory
- System uptime

5. File System Module

- Create a file named `employee.txt` in `employee` folder.
- Write employee information into it (name, email, salary, department) and read it back.

6. NPM Practice

Create a `package.json`, install the `nodemon`, `mongoose` package, and explain what changes in the project folder after installation.

7. MongoDB CRUD with Validation

Perform MongoDB CRUD operations for the Employee domain using a Mongoose schema with the fields below.

Field Name	Data Type	Description	Validation
Name	String	Employee Name	Required
Email	String	Employee Email	Required, unique
department	String	Department Name	Required, must be one of: "HR", "IT", "Sales", "Finance", "Marketing"
age	Number	Employee Age	Required, minimum 18, maximum 60
city	String	Employee City	Required
salary	Number	Monthly Salary	Required, minimum 10000, maximum 50000
isActive	Boolean	Employment Status	Default value true

Operations using User Defined Functions:

1. Add a new employee
2. View all employees
3. Delete multiple employees
4. Update employee details
5. Try to add an employee with a missing required field, and display the validation error message.
6. Try to add an employee with age below 18, salary below 10000, or a department not in the allowed list, and display the validation error messages.
7. Try to add two employees with the same email, and display the duplicate key error.
8. Try to update an employee's age to 15 using updateOne().

Part 2: MongoDB Test (Restaurant_DB)

8. Set up a MongoDB database named "Restaurant_DB".
9. Create a collection named "menu" within the "Restaurant_DB" database.
10. Insert at least 5 documents into the "menu" collection, each representing a dish with fields such as dishName, category, price, and availableQuantity. Ensure that each document has a unique identifier.
11. Retrieve and display all documents from the "menu" collection.
12. Find and display dishes whose price is between ₹200 and ₹600.
13. Find and display dishes belonging to a specific category (e.g., "Starters").
14. Update the price of a particular dish.
15. Increment the availableQuantity of dishes that are currently in stock by 2.
16. Delete a dish document from the collection.
17. Delete all dishes that have 0 availableQuantity.

Part 3: MongoDB Test (Gym_DB)

18. Set up a MongoDB database named "Gym_DB" and create a collection named "members" within it.
19. Insert at least 5 documents into the "members" collection, each representing a member with fields such as name, age, membershipPlan, trainerAssigned, and city. Ensure that each member has a unique identifier.
20. Retrieve and display all documents from the "members" collection.
21. Find and display members whose age is between 20 and 40 years old.
22. Find and display members enrolled in a specific plan (e.g., "Premium").
23. Update the trainerAssigned field of a member.
24. Increment the age of members above 50 years by 1 year.
25. Delete all members belonging to a specific city.

Part 4: MongoDB Advanced Queries (use the "members" collection)

26. Fetch only one document from the collection using findOne().
27. Find members whose age is greater than 40.
28. Retrieve documents sorted in ascending order of age.
29. Find members whose age is less than 25.
30. Find documents using multiple conditions with \$and, and then using the \$or condition.